

Sistemas Operacionais

Roni Francis Shigueta

SINCRONIZAÇÃO E COMUNICAÇÃO ENTRE PROCESSOS

CONTEÚDO

- Introdução
- Aplicações concorrentes
- Especificação de concorrência em programas
- Problemas de compartilhamento de recursos
- Exclusão mútua

INTRODUÇÃO

Efeito colateral da multiprogramação:

Um sistema multiprogramável (concorrente) pode resultar em inconsistências.

Ex.: um processo tenta ler uma área de memória que outro processo está escrevendo.

-> **Necessidade de sincronização.**

APLICAÇÕES CONCORRENTES

- As aplicações concorrentes necessitam se comunicar.
- Formas de comunicação concorrente: **variáveis compartilhadas** na memória ou **troca de mensagens**
- Mecanismo responsável por garantir que cada processo será executado quando o recurso estiver pronto: **sincronização**

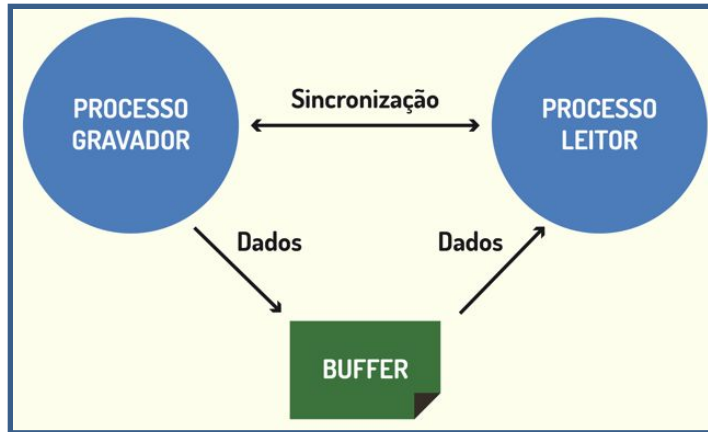


Figura 1 –Processo gravador e leitor

Fonte: <https://bit.ly/2XGYB8o>

ESPECIFICAÇÃO DE CONCORRÊNCIA EM PROGRAMAS (FORK E JOIN)

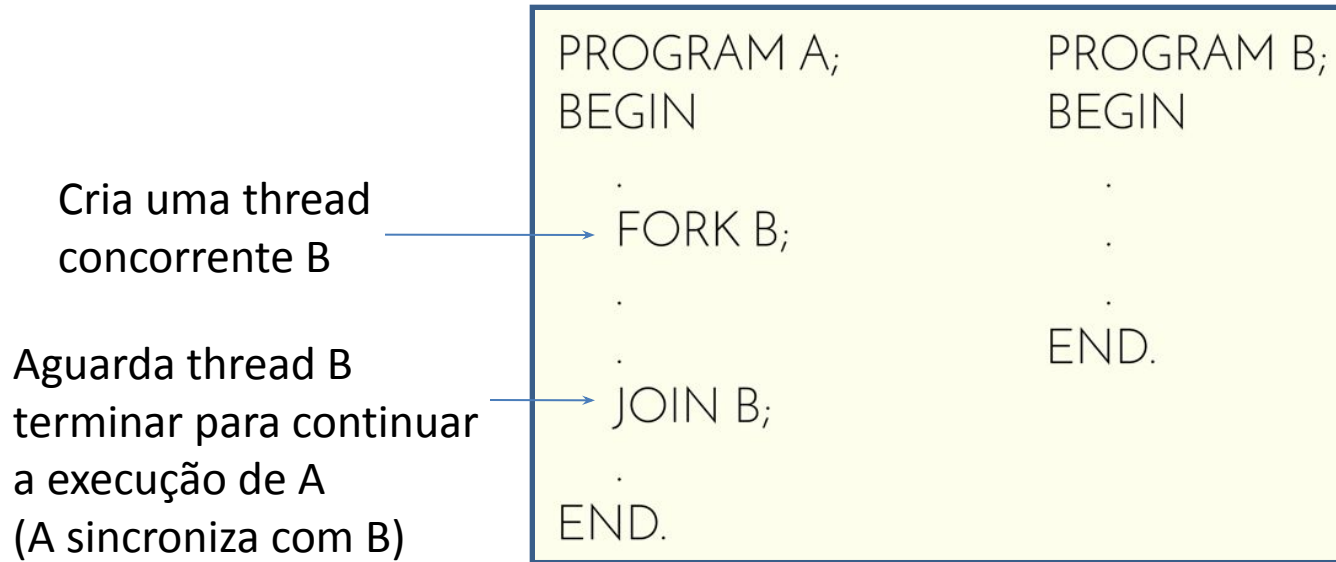


Figura 2 – Fork e join

Fonte: <https://bit.ly/2XGYB8o>

ESPECIFICAÇÃO DE CONCORRÊNCIA EM PROGRAMAS (PARBEGIN E PAREND)

- PARBEGIN: comandos subsequentes são executados de forma concorrente. Novos processos e novas threads são criadas.
- PAREND: ponto de sincronização. O processamento aguardará que todos os processos e threads do PARBEGIN anterior sejam finalizados.

PROBLEMAS DE COMPARTILHAMENTO DE RECURSOS

Alguns potenciais problemas da concorrência:

- Compartilhamento de arquivos
- Compartilhamento de memória principal por meio de uma variável

PROBLEMA COM COMPARTILHAMENTO DE ARQUIVOS

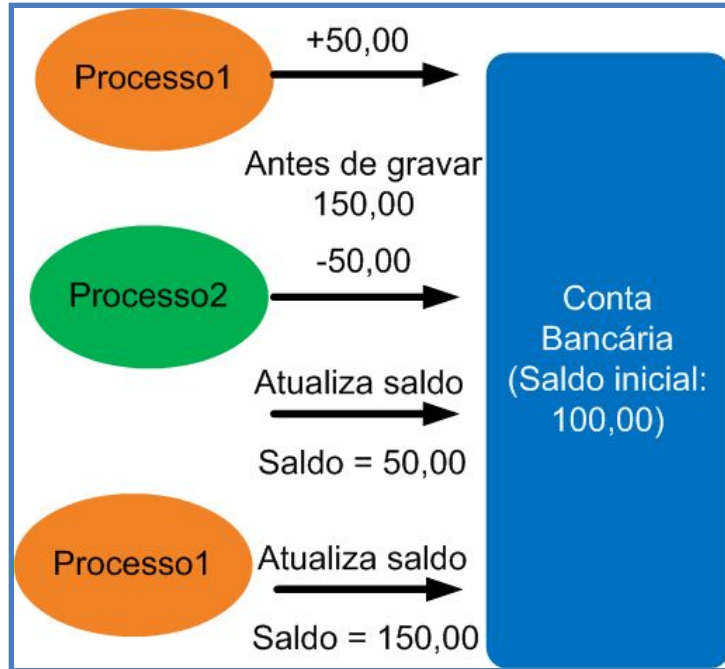


Figura 3 – Problema do compartilhamento de arquivos

- Resultado final depende de quem roda por último.

PROBLEMA COM COMPARTILHAMENTO DE VARIÁVEL NA MEMÓRIA PRINCIPAL

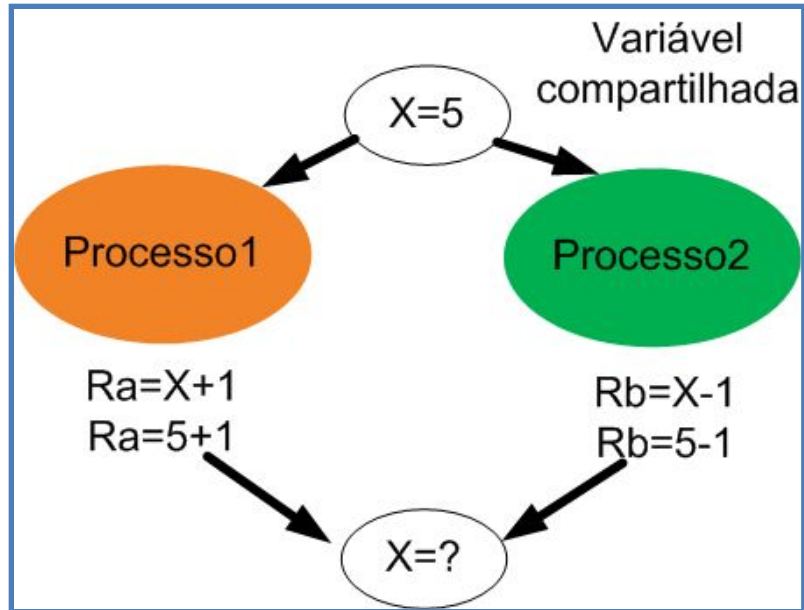


Figura 4 – Problema de compartilhamento de variável

- Resultado final depende de quem grava por último.

EXCLUSÃO MÚTUA

- Tem por objetivo evitar que dois processos acessem o mesmo recurso compartilhado ao mesmo tempo.
- Parte do código que acessa o recurso compartilhado: região crítica.
- Evitar que dois processos entrem na região crítica ao mesmo tempo.

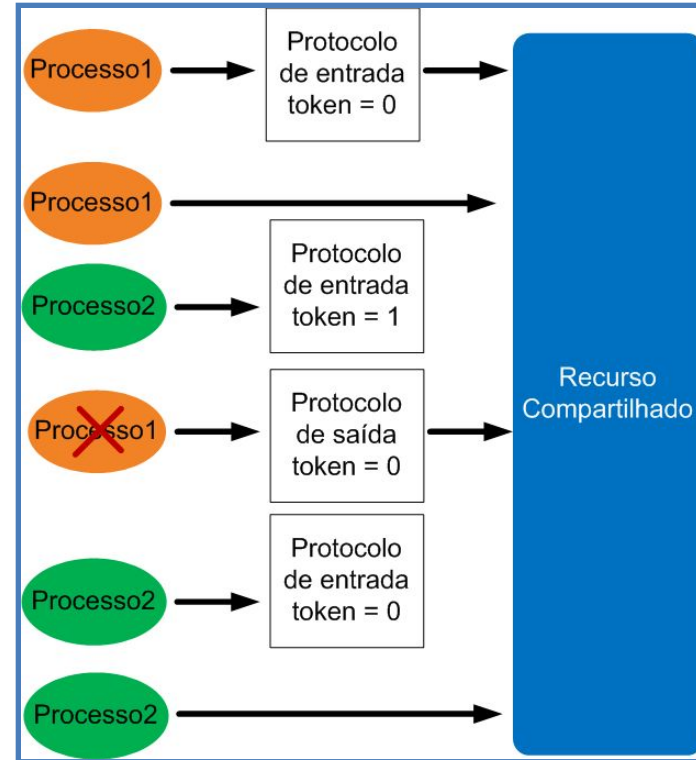
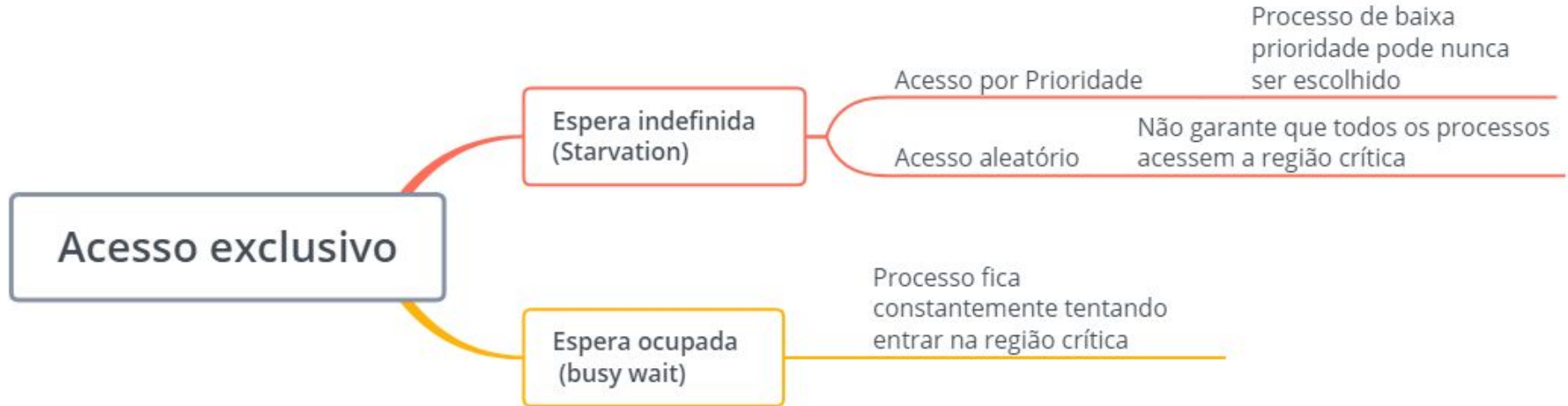


Figura 5 – Exclusão mútua

PROBLEMA DO ACESSO EXCLUSIVO AO RECURSO COMPARTILHADO



SOLUÇÕES PARA IMPLEMENTAR A EXCLUSÃO MÚTUA

Soluções

Hardware

Desabilitar interrupções quando entra na região crítica e habilita quando sai

Problema se processo não habilita a interrupção novamente

Instrução Test-And-Set (TAS)

Instrução de máquina que permite acessar uma área que não sofre interrupção

Problema da espera indefinida

Software

Solução de Peterson

Utiliza 2 variáveis `flag[0]` e `turn`

`flag` define se o processo quer entrar na região crítica

`turn` representa se a região crítica está ocupada ou não

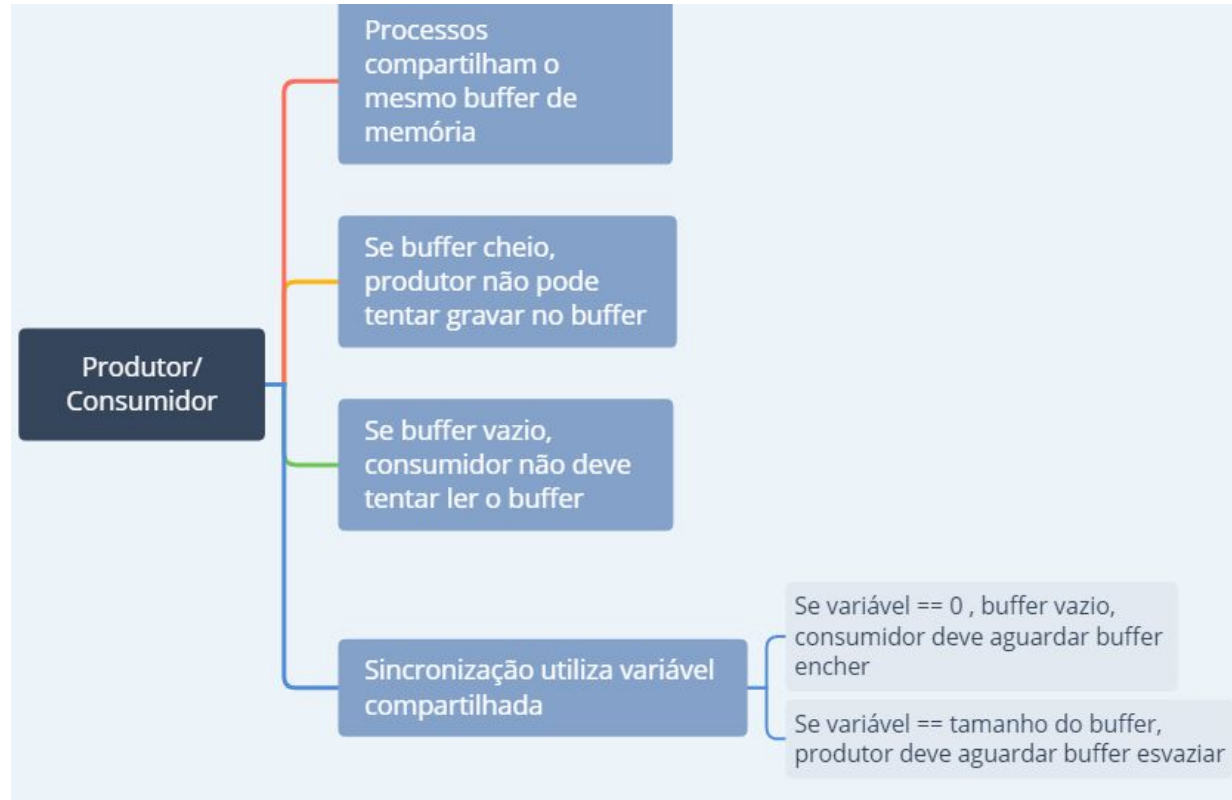
Se `flag[0] == 1` && `turn == 0`, espera liberar a região crítica

Problema de espera ocupada

Solução:
mecanismos de sincronização

SINCRONIZAÇÃO CONDICIONAL

- ❑ Ocorre quando dois processos precisam sincronizar-se para acessar o recurso compartilhado.
- ❑ A sincronização é feita por condições de acesso, ou seja o processo deve aguardar o recurso ser liberado.
- ❑ Ex.: problema do produtor/consumidor



REFERÊNCIAS

TANENBAUM, A. S. **Sistemas operacionais modernos.** São Paulo: Pearson Prentice Hall, 2009.

DEITEL, H. M; DEITEL, P. J.; DEITEL, D. R. C. **Sistemas Operacionais.** São Paulo: Pearson Prentice Hall, 2005.